

COMPSCI 689

Lecture 12: Neural Network Architectures

Part 1

Benjamin M. Marlin

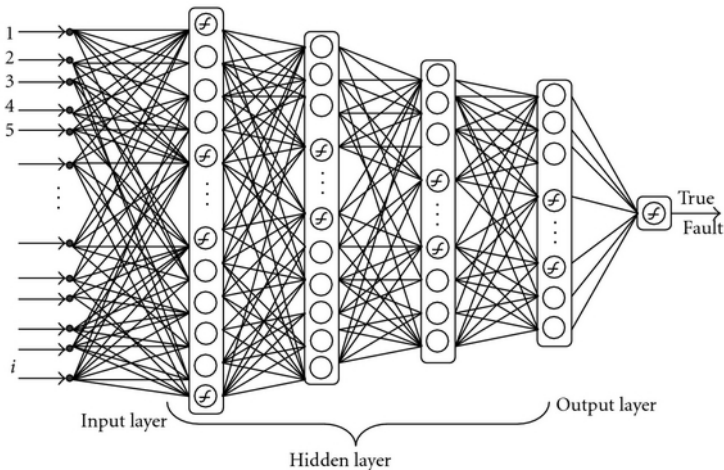
College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Introduction

- Basic MLPs have multiple layers of hidden units that are fully connected with distinct weights on each connection.
- Such an architecture does not encode any domain knowledge about the relationship between the inputs and outputs.

Multi-Layer Perceptron



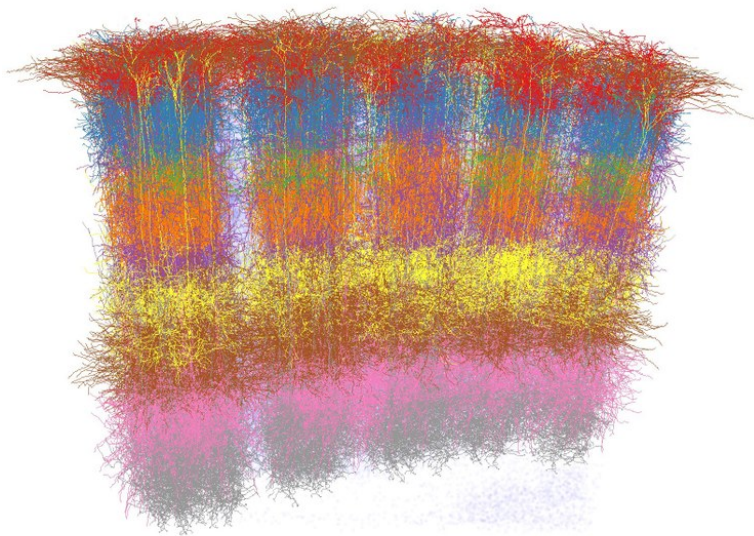
Introduction

- For specialized types of inputs like images, sequences or graphs, it is possible to encode domain knowledge about the structure of the inputs or input-output relationships into the architecture of a neural network.
- This can help make learning more efficient by using notions like sparsity and invariance to reduce the number of parameters that must be learned.

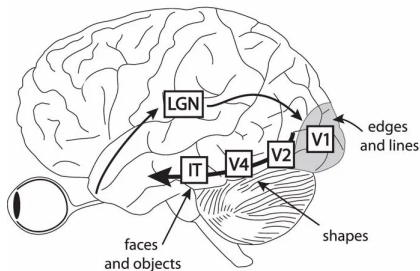
Images and Sequences

- Two most common types of structures encountered in the neural networks literature are images and sequences.
- In object classification, an object in an image can occur at any scale, orientation, location, etc., and the information about whether an image contains an object is typically spatially localized.
- In sequence data like text, the underlying grammar and vocabulary create regularities in sequences in terms of both short and long range dependencies between characters and words.

Cortical Columns: Local Processing



Visual Areas: Hierarchical Receptive Fields



Receptive fields size

IT



V4



V2



V1



visual field

Features



faces



objects



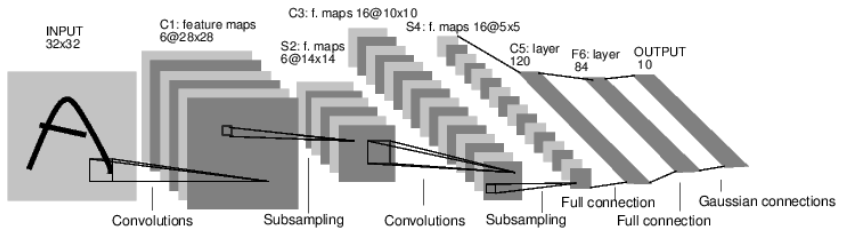
shapes

edges
and lines

The Brain and CNNs

- Deep learning for images uses a modified deep architecture called a *convolutional network* or convnet or CNN.
- This architecture was popularized in the 1980s and was directly inspired by the structure of visual processing in the brain.

The CNN Architecture



CNN Invariances

- The convolutional network architecture encodes two primary pieces of domain knowledge: translational invariance and spatial locality.
- The architecture uses *max pooling* to make individual hidden units more invariant to small local translations in an image.
- The architecture achieves translational invariance more globally by using the same set of weights in every image patch via *convolution*.
- The basic network architecture consists of a number of convolution, non-linearity, pooling stages followed by several fully connected layers.

Convolution

- Given a function $f(i, j)$ and a convolution kernel $g(k, l)$, the convolution of f and g is:

$$(f * g)(i, j) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(i - k, j - l) g(k, l) dk dl$$

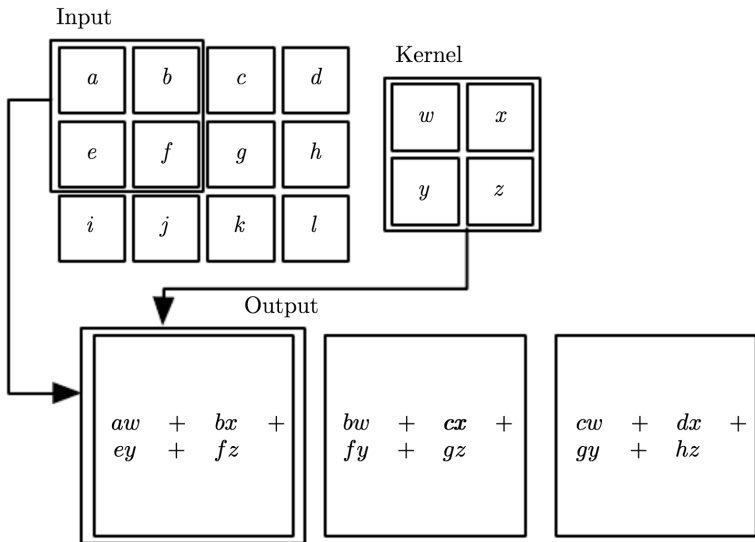
Convolution and Cross-Correlation

- Convolutional neural network models actually implement a discrete version of a closely related operations called “cross-correlation.”
- Given a tensor $\mathbf{x}$ of shape (H, W, C) and a kernel w_k of shape (L, M, C) for $L \leq H, M \leq W$, we have:

$$c[i, j, k] = \sum_{l=1}^L \sum_{m=1}^M \sum_{c=1}^C x[i+l, j+m, c] w[l, m, c]$$

- Here H and W represent the height and width of the input. C represents the number of input *channels* (e.g., 3 for a RGB image). L and M represent the width and height of the kernel.

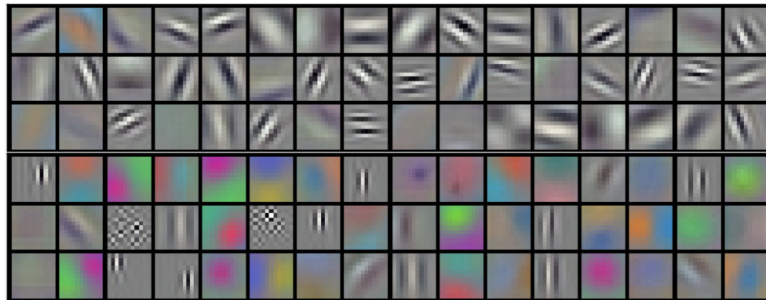
Example: Single Channel 2D Cross-Correlation



The Convolutional Layer

- For 2D CNNs, each convolutional layer contains several kernels $\mathbf{w}_1, \dots, \mathbf{w}_K$.
- Suppose the input to a convolution layer has shape (H, W, C) . The output of a convolutional layer using K kernels of shape (L, M, C) has shape $(H - L + 1, W - M + 1, K)$.
- In standard CNN architectures, it is common to increase the number of channels as we move from the input to the output.
- The use of small convolution kernels (e.g., $5 \times 5 \times C$) results in a massive reduction in the number of weights relative to an MLP and creates hidden units that have spatially localized inputs.

Example Layer 1 Filters/Kernels (Natural Images)



Example Filtermaps (Natural Images)

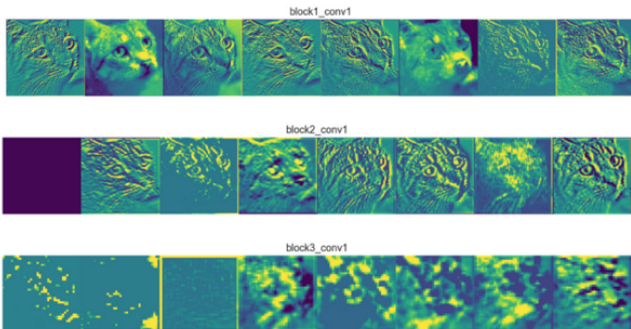
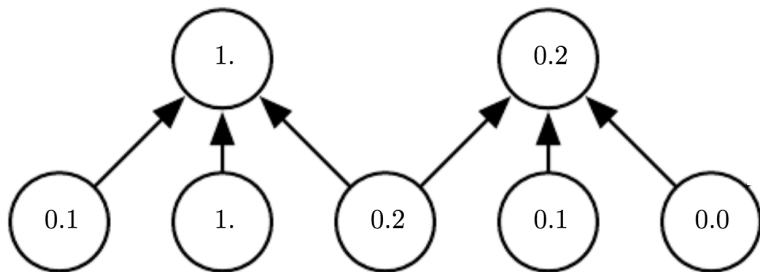


Image credit: Arden Dertat

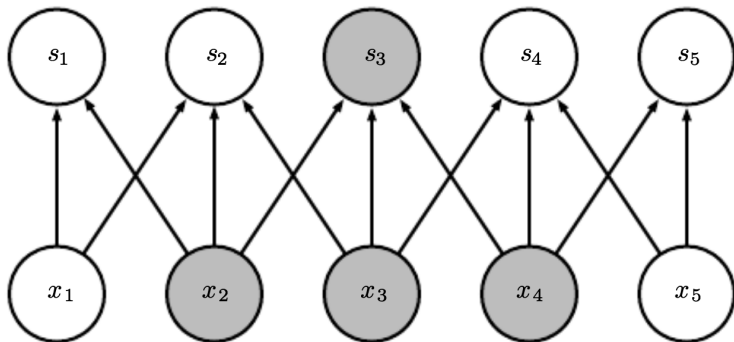
Pooling

- The pooling operation effectively sub-samples a feature map by aggregating over (possibly overlapping) blocks of the image.
- The most common pooling operation is *max pooling*.
- Max pooling replaces the values in each block of a feature map with their maximum.

Example: 1D Pooling

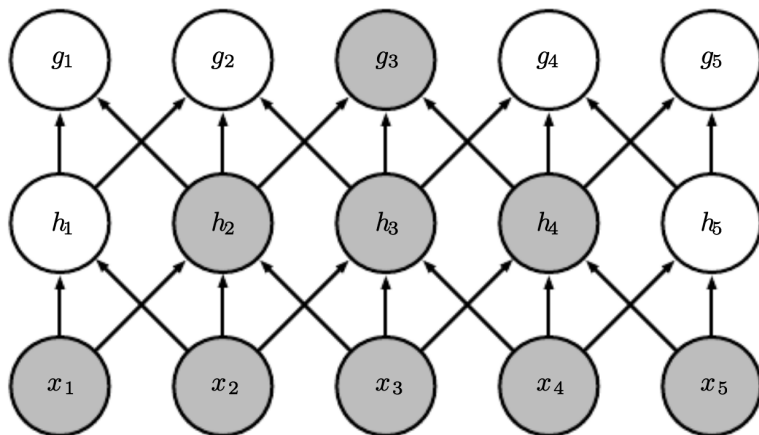


Example: 1D Receptive Field

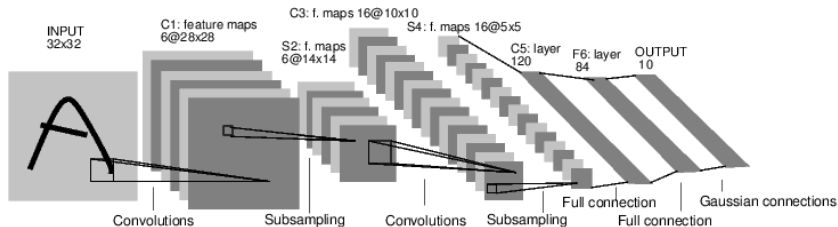


Via analogy to animal visual systems, the set of inputs that feed into a given hidden unit in a given feature map are referred to as that unit's *receptive field*.

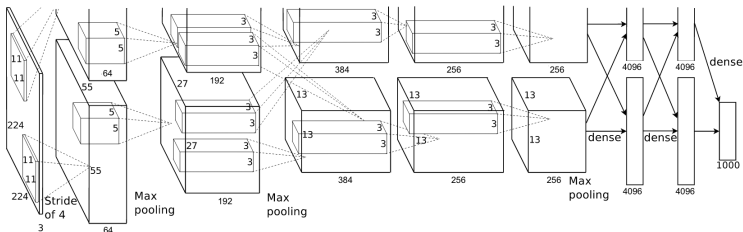
Example: 1D Receptive Field



LeNet5 (1995)



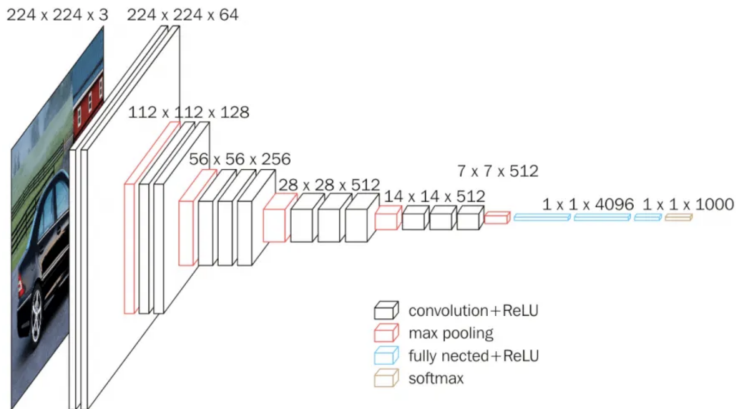
AlexNet (2012)



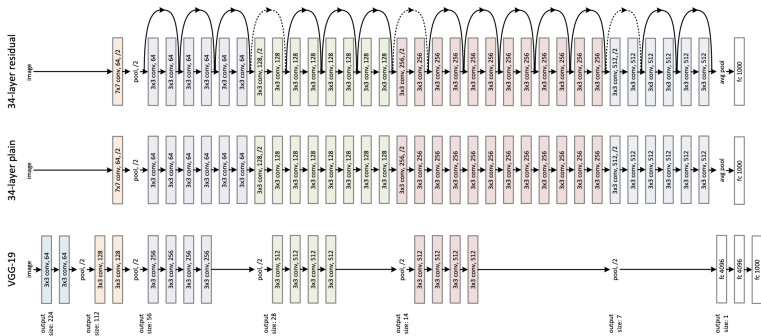
```
self.features = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(64, 128, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(128, 192, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),
    nn.Conv2d(192, 128, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),
    nn.Conv2d(128, 192, kernel_size=3, padding=1),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
)
```

```
self.classifier = nn.Sequential(
    nn.Dropout(),
    nn.Linear(256 * 6 * 6, 4096),
    nn.ReLU(inplace=True),
    nn.Dropout(),
    nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),
    nn.Linear(4096, num_classes),
)
```

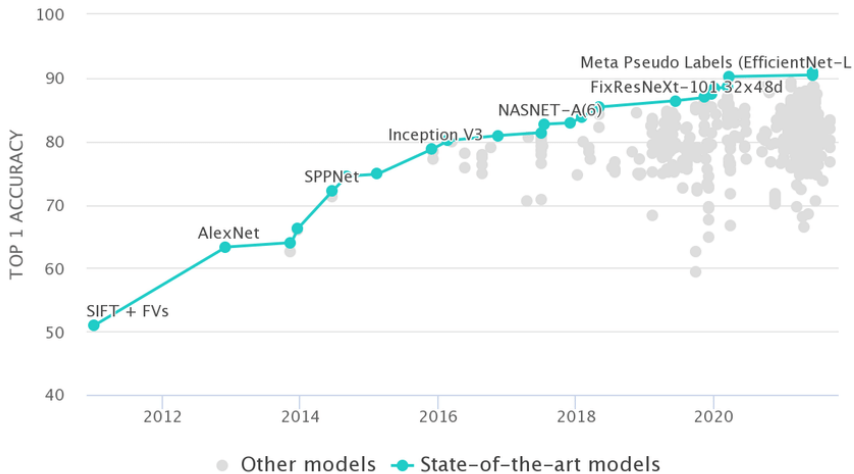
VGG19 (2014)



ResNet34 (2015)



ImageNet Benchmark



Pre-Trained Models

Transforming and augmenting images

TVTensors

● Models and pre-trained weights

Datasets

Utils

Operators

Decoding / Encoding images and videos

Feature extraction for model inspection

Examples and training references

Examples and tutorials

Training references

PyTorch Libraries

PyTorch

torchaudio

torchtext

torchvision

TorchElastic

TorchServe

PyTorch on XLA Devices

Table of all available classification weights

Accuracies are reported on ImageNet-1K using single crops:

Weight	Acc@1	Acc@5	Params	GFLOPS	Recipe
AlexNet_Weights.IMAGENET1K_V1	56.522	79.066	61.1M	0.71	link
ConvNeXt_Base_Weights.IMAGENET1K_V1	84.062	96.87	88.6M	15.36	link
ConvNeXt_Large_Weights.IMAGENET1K_V1	84.414	96.976	197.8M	34.36	link
ConvNeXt_Small_Weights.IMAGENET1K_V1	83.616	96.65	50.2M	8.68	link
ConvNeXt_Tiny_Weights.IMAGENET1K_V1	82.52	96.146	28.6M	4.46	link
DenseNet121_Weights.IMAGENET1K_V1	74.434	91.972	8.0M	2.83	link
DenseNet161_Weights.IMAGENET1K_V1	77.138	93.56	28.7M	7.73	link
DenseNet169_Weights.IMAGENET1K_V1	75.6	92.806	14.1M	3.36	link
DenseNet201_Weights.IMAGENET1K_V1	76.896	93.37	20.0M	4.29	link
EfficientNet_B0_Weights.IMAGENET1K_V1	77.692	93.532	5.3M	0.39	link
EfficientNet_B1_Weights.IMAGENET1K_V1	78.642	94.186	7.8M	0.69	link
EfficientNet_B1_Weights.IMAGENET1K_V2	79.838	94.934	7.8M	0.69	link
EfficientNet_B2_Weights.IMAGENET1K_V1	80.608	95.31	9.1M	1.09	link
EfficientNet_B3_Weights.IMAGENET1K_V1	82.008	96.054	12.2M	1.83	link
EfficientNet_B4_Weights.IMAGENET1K_V1	83.384	96.594	19.3M	4.39	link
EfficientNet_B5_Weights.IMAGENET1K_V1	83.444	96.628	30.4M	10.27	link
EfficientNet_B6_Weights.IMAGENET1K_V1	84.008	96.916	43.0M	19.07	link
EfficientNet_B7_Weights.IMAGENET1K_V1	84.122	96.908	66.3M	37.75	link
EfficientNet_V2_L_Weights.IMAGENET1K_V1	85.808	97.788	118.5M	56.08	link
EfficientNet_V2_M_Weights.IMAGENET1K_V1	85.112	97.156	54.1M	24.58	link
EfficientNet_V2_S_Weights.IMAGENET1K_V1	84.228	96.878	21.5M	8.37	link
GoogLeNet_Weights.IMAGENET1K_V1	69.778	89.53	6.6M	1.5	link
Inception_V3_Weights.IMAGENET1K_V1	77.294	93.45	27.2M	5.71	link
MNASNet0_5_Weights.IMAGENET1K_V1	67.734	87.49	2.2M	0.1	link
MNASNet0_75_Weights.IMAGENET1K_V1	71.18	90.496	3.2M	0.21	link
MNASNet1_0_Weights.IMAGENET1K_V1	73.456	91.51	4.4M	0.31	link

Models and pre-trained weights:

+ General information on pre-trained weights

+ Classification

+ Semantic Segmentation

+ Object Detection, Instance Segmentation and Person Keypoint Detection

+ Video Classification

Optical Flow